

Software Requirements Specification

Scholarship Management and Verification Portal

Version 1.0

Prepared for Hackathon Submission

Category: Vibe Coding & Full Stack Development

Date: [Insert Date]

Table of Contents

Table of Contents.....	2
1. Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Definitions, Acronyms, and Abbreviations	4
1.4 References	5
1.5 Overview	5
2. Overall Description.....	6
2.1 Product Perspective	6
2.2 Product Functions (Summary)	6
2.3 User Classes and Characteristics	6
2.4 Operating Environment	6
2.5 Design and Implementation Constraints	6
2.6 Assumptions and Dependencies	7
3. System Features (Functional Requirements)	8
3.1 Authentication & Access Control Module.....	8
3.2 Student (Applicant) Module	8
3.3 Staff (Verification) Module	8
3.4 Admin (Management) Module	9
3.5 Workflow & Status Engine.....	9
3.6 Notification Module.....	9
3.7 Analytics & Reporting Module.....	10
3.8 Audit Log Module	10
3.9 Institutional Integration Module (Simulated)	10
4. External Interface Requirements	11
4.1 User Interfaces	11
4.2 Hardware Interfaces.....	11
4.3 Software Interfaces	11
4.4 Communication Interfaces.....	11
5. Non-Functional Requirements.....	12
5.1 Performance Requirements.....	12
5.2 Security Requirements.....	12
5.3 Reliability & Availability	12
5.4 Usability	12
5.5 Maintainability & Scalability.....	12
5.6 Portability	12
6. Other Requirements	13

6.1 Data Requirements.....13

6.2 Legal and Compliance Considerations13

6.3 Future Enhancements (Out of Current Scope).....13

6.4 Approval.....13

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Scholarship Management and Verification Portal. It is intended to guide the design, development, and testing of the system, and to serve as an agreed reference between the development team and evaluators regarding what the system will and will not do.

1.2 Scope

The Scholarship Management and Verification Portal is a web-based application that digitizes the end-to-end scholarship lifecycle: application submission, document verification, approval, and disbursement. The system provides three role-based access levels — Student (Applicant), Staff (Verifier), and Admin (Administrator) — and automates workflow routing, sends real-time notifications, and generates analytical reports. The system integrates with institutional systems (student records, identity verification, disbursement) via a mocked/simulated interface layer for the purposes of this build.

In scope for this version:

- Role-based registration, login, and access control
- Scholarship scheme creation and management
- Online application submission with document upload
- Staff-level document verification and eligibility checking
- Admin-level final approval, override, and disbursement tracking
- Automated workflow state transitions
- In-app and email notifications
- Analytics dashboard with visual reports
- Audit logging of all critical actions

Out of scope for this version:

- Real integration with live institutional ERP, government ID, or banking systems (simulated only)
- SMS-based notifications
- Mobile native applications (web-responsive only)
- Multi-language localization

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
RBAC	Role-Based Access Control
JWT	JSON Web Token, used for authentication
Admin	System administrator responsible for scheme configuration and final approvals
Staff	Verification officer responsible for document review
Student	Applicant applying for a scholarship

Term	Definition
Scheme	A specific scholarship program with its own eligibility criteria and budget
API	Application Programming Interface
CRUD	Create, Read, Update, Delete

1.4 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications
- Project problem statement: Scholarship Management and Verification Portal (Hackathon Brief)
- MongoDB, Express.js, React, Node.js official documentation

1.5 Overview

Section 2 describes the overall product context, users, and constraints. Section 3 lists detailed functional requirements grouped by module. Section 4 describes external interface requirements. Section 5 defines non-functional requirements. Section 6 covers data and other supporting requirements.

2. Overall Description

2.1 Product Perspective

The portal is a standalone, self-contained web application composed of a React-based frontend and a Node.js/Express backend, backed by a MongoDB database. It replaces manual, paper- and spreadsheet-based scholarship processing with a unified digital workflow. It is designed so that its mocked institutional-integration layer can later be replaced with real external system connections without altering the core workflow logic.

2.2 Product Functions (Summary)

- User registration, authentication, and role-based access
- Scholarship scheme creation and lifecycle management
- Application submission with document upload
- Document verification and eligibility auto-checking
- Configurable, automated approval workflow (staff-only or staff-plus-admin)
- Real-time status tracking for applicants
- In-app and email notifications
- Analytical dashboards and reports
- Full audit trail of system actions

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
Student	Applies for scholarships, uploads documents, tracks status	Low — general public users
Staff	Verifies documents, checks eligibility, recommends decisions	Low to moderate — institutional staff
Admin	Configures schemes, gives final approval, views analytics, manages staff	Moderate — administrative personnel

2.4 Operating Environment

- Client: Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop or mobile, screen widths from 375px upward
- Server: Node.js runtime (v18+), hosted on a cloud platform (e.g., Render/Railway)
- Database: MongoDB (self-hosted or Atlas cloud)
- Frontend hosting: Static hosting/CDN (e.g., Vercel)

2.5 Design and Implementation Constraints

- Must be built and demo-ready within the hackathon time frame (2-day development cycle)
- Institutional system integrations are simulated/mocked, not live
- File uploads limited to PDF/JPG/PNG formats, maximum 5 MB per file
- Must use free-tier or open-source infrastructure suitable for a hackathon budget

2.6 Assumptions and Dependencies

- Users have access to a stable internet connection and a modern browser
- Institutional identity/document data used for mock verification is provided as seed/sample data
- Email notifications depend on a functioning SMTP provider (e.g., via Nodemailer)
- Cloud hosting providers (Vercel, Render, MongoDB Atlas) remain available during development and demo

3. System Features (Functional Requirements)

3.1 Authentication & Access Control Module

ID	Requirement
FR-1.1	The system shall allow Students, Staff, and Admins to register and log in with unique credentials.
FR-1.2	The system shall hash all passwords before storage (bcrypt or equivalent).
FR-1.3	The system shall issue a signed JWT on successful login containing the user's role.
FR-1.4	The system shall restrict access to role-specific routes and API endpoints based on the authenticated user's role, enforced on both client and server.
FR-1.5	The system shall allow users to view and edit their own basic profile information.

3.2 Student (Applicant) Module

ID	Requirement
FR-2.1	The system shall display a list of active scholarship schemes with title, amount, deadline, and eligibility summary.
FR-2.2	The system shall allow a student to view full details of a scholarship scheme before applying.
FR-2.3	The system shall provide an application form capturing academic details, income, and category information.
FR-2.4	The system shall allow students to upload required supporting documents against the application.
FR-2.5	The system shall validate uploaded files by type and size before accepting them.
FR-2.6	The system shall lock an application from further editing once submitted.
FR-2.7	The system shall display a real-time status tracker showing the application's current stage.
FR-2.8	The system shall allow a student to re-upload only the specific document(s) flagged as invalid, without resubmitting the entire application.
FR-2.9	The system shall allow students to view a history of all their past and current applications.

3.3 Staff (Verification) Module

ID	Requirement
FR-3.1	The system shall display a queue of applications assigned to the logged-in staff member.
FR-3.2	The system shall allow staff to view an applicant's submitted details and documents together.
FR-3.3	The system shall allow staff to mark each document individually as Valid, Invalid, or Needs Resubmission, with an optional remark.
FR-3.4	The system shall automatically evaluate an application against the scheme's eligibility criteria and display a pass/fail indicator with the specific failing condition, if any.

ID	Requirement
FR-3.5	The system shall allow staff to submit a recommendation of Approve or Reject for an application.
FR-3.6	The system shall route a staff recommendation either to auto-approval or to the Admin approval queue, based on the scheme's configured approval requirement.
FR-3.7	The system shall automatically return an application to the staff queue if a previously flagged document is resubmitted by the student.
FR-3.8	The system shall display basic performance statistics (applications processed, average turnaround) to the staff member.

3.4 Admin (Management) Module

ID	Requirement
FR-4.1	The system shall allow Admin to create a scholarship scheme, defining eligibility rules, amount, seat count, deadline, and whether admin approval is required.
FR-4.2	The system shall allow Admin to edit or close an existing scheme.
FR-4.3	The system shall display a queue of applications awaiting final admin approval.
FR-4.4	The system shall allow Admin to approve or reject an application recommended by staff.
FR-4.5	The system shall allow Admin to override a staff recommendation, provided a justification is entered; the override shall be recorded in the audit log.
FR-4.6	The system shall allow Admin to mark an approved application as disbursed.
FR-4.7	The system shall allow Admin to add, remove, and view staff accounts.
FR-4.8	The system shall allow Admin to view and, if necessary, deactivate student accounts.

3.5 Workflow & Status Engine

ID	Requirement
FR-5.1	The system shall move every application through a defined state machine: Submitted, Under Review, Verified, Pending Admin Approval (conditional), Approved, Rejected, Disbursed.
FR-5.2	The system shall automatically assign a newly submitted application to a staff member or staff pool.
FR-5.3	The system shall automatically close a scholarship scheme once its deadline has passed or seat capacity is reached.

3.6 Notification Module

ID	Requirement
FR-6.1	The system shall generate an in-app notification for the relevant user on every application status change.

ID	Requirement
FR-6.2	The system shall send an email notification for key events: submission, document rejection, approval, rejection, and disbursement.
FR-6.3	The system shall display an unread notification count and allow users to mark notifications as read.

3.7 Analytics & Reporting Module

ID	Requirement
FR-7.1	The system shall display the breakdown of applications by status as a chart.
FR-7.2	The system shall display application volume over time as a trend chart.
FR-7.3	The system shall display approval rate per scholarship scheme.
FR-7.4	The system shall display fund utilization (disbursed amount vs. total budget) per scheme.
FR-7.5	The system shall display the most common reasons for document rejection.

3.8 Audit Log Module

ID	Requirement
FR-8.1	The system shall record every approval, rejection, override, and disbursement action with actor, timestamp, and details.
FR-8.2	The system shall provide Admin with a searchable and filterable view of the audit log.
FR-8.3	The system shall not allow audit log entries to be edited or deleted through the application interface.

3.9 Institutional Integration Module (Simulated)

ID	Requirement
FR-9.1	The system shall provide a mock service to validate a student's identity/enrollment number against a sample institutional record.
FR-9.2	The system shall provide a mock disbursement confirmation service simulating a bank transfer.
FR-9.3	The system shall isolate all mock integration calls behind a service layer that can be replaced with a real external API without modifying core workflow logic.

4. External Interface Requirements

4.1 User Interfaces

- Responsive web interface built with React and Tailwind CSS, supporting desktop and mobile browser widths
- Three distinct dashboard layouts: Student, Staff, and Admin, each showing only role-relevant navigation and data
- Consistent status indicators using color-coded badges across all dashboards

4.2 Hardware Interfaces

None. The system is a standard web application with no direct hardware dependency; it runs on any device capable of running a modern web browser.

4.3 Software Interfaces

Interface	Description
Database	MongoDB via Mongoose ODM for all persistent data storage
Authentication	JWT-based token issuance and verification
File Storage	Multer for handling multipart file uploads; local or cloud object storage
Email Service	SMTP integration via Nodemailer for outbound notification emails
Mock Institutional Systems	Internal simulated services standing in for external ERP/ID/bank APIs

4.4 Communication Interfaces

- HTTPS for all client-server communication
- RESTful JSON APIs exposed by the backend and consumed by the frontend

5. Non-Functional Requirements

5.1 Performance Requirements

- Application list and queue views shall be paginated to avoid loading the entire dataset at once
- Dashboard analytics queries shall return results within an acceptable time for interactive use (target: under 2 seconds for aggregated data at hackathon-scale test data volumes)

5.2 Security Requirements

- All passwords shall be stored as salted hashes, never in plain text
- All role-restricted API endpoints shall verify the user's role server-side, independent of frontend checks
- Students shall only be able to access their own application data; this shall be enforced at the database query level, not merely hidden in the UI
- File uploads shall be validated for type and size to reduce risk of malicious uploads

5.3 Reliability & Availability

- The workflow engine shall ensure an application cannot skip a required status transition (e.g., cannot move directly from Submitted to Disbursed)
- Every critical action (approve, reject, override, disburse) shall be logged for traceability

5.4 Usability

- The interface shall present a visual status tracker so non-technical users (students) can understand application progress without instructions
- Error and validation messages shall be clear and specific (e.g., naming the exact missing field or failed eligibility rule)

5.5 Maintainability & Scalability

- The codebase shall be organized into clearly separated modules (per Section 3) to allow independent development and future extension
- The mock institutional integration layer shall be replaceable with real external APIs without requiring changes to workflow or approval logic

5.6 Portability

- The frontend shall function correctly on the latest versions of major browsers (Chrome, Firefox, Edge, Safari) without browser-specific code paths

6. Other Requirements

6.1 Data Requirements

Core entities maintained by the system:

- Users — student, staff, and admin accounts with role, credentials, and profile data
- Scholarships — scheme definitions including eligibility criteria, amount, deadline, and approval configuration
- Applications — submitted applications, linked documents, status history, and remarks
- Notifications — per-user notification records with read/unread state
- Audit Logs — immutable records of critical system actions

6.2 Legal and Compliance Considerations

- Applicant personal and financial data (income, ID details) shall be treated as sensitive and access-restricted to authorized roles only
- For a production deployment, the system would need to align with applicable data protection regulations; this is noted as a future consideration beyond hackathon scope

6.3 Future Enhancements (Out of Current Scope)

- Live integration with real institutional ERP, national ID, and banking/disbursement systems
- SMS notification channel
- AI-assisted document authenticity checks
- Multi-language support

6.4 Approval

This SRS document is intended for internal team alignment and hackathon evaluation purposes. It should be revisited and revised as requirements evolve during development.